

Εργαστήριο Βάσεων Δεδομένων

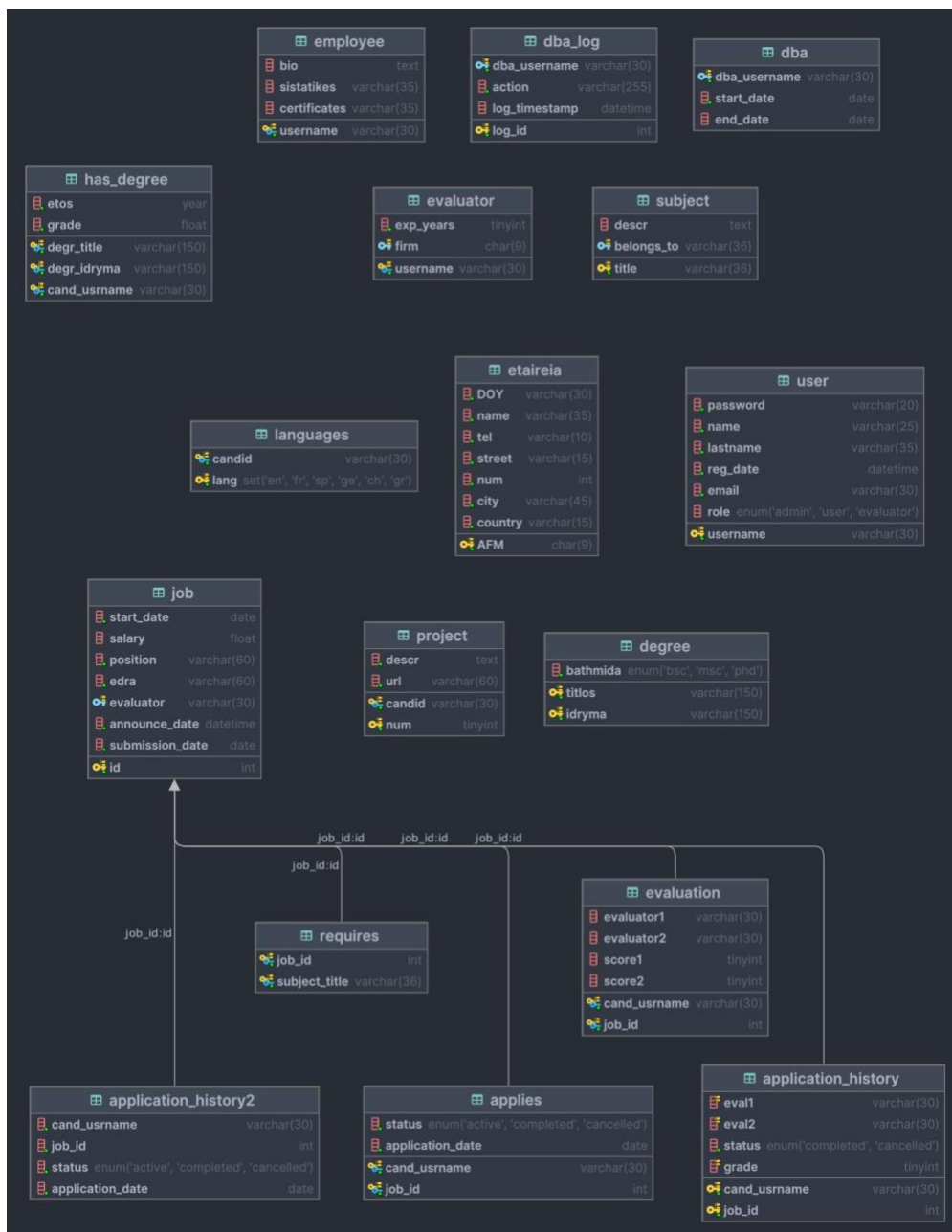
Εργαστηριακή Άσκηση 2023-2024

Χρυσοβαλάντης Γκόρο 1054356

Χαράλαμπος Γιαννέλης 1093341

1. Σχεδιασμός βάσης δεδομένων

Κατασκευάστηκε η βάση δεδομένων όπως περιγράφηκε στο αρχείο της προπαρασκευαστικής φάσης. Ο κώδικας που χρησιμοποιήθηκε για την κατασκευή των πινάκων αποτυπώνεται στο παράρτημα. Ακολουθεί το ER-Schema της βάσης.



Κώδικας υλοποίησης κεφαλαίου 1

```
-- Table 'user'
CREATE TABLE user (
    username VARCHAR(30) PRIMARY KEY UNIQUE NOT NULL,
    password VARCHAR(20) NOT NULL,
    name VARCHAR(25) NOT NULL,
    lastname VARCHAR(35) NOT NULL,
    reg_date DATETIME NOT NULL,
    email VARCHAR(30) NOT NULL,
    role ENUM('admin','user','evaluator') DEFAULT 'user'
);
```

```
--Table 'etairia'
CREATE TABLE etaireia (
    AFM CHAR(9) PRIMARY KEY NOT NULL,
    DOY VARCHAR(30) NOT NULL,
    name VARCHAR(35) NOT NULL,
    tel VARCHAR(10) NOT NULL,
    street VARCHAR(15) NOT NULL,
    num INT(11) NOT NULL,
    city VARCHAR(45) NOT NULL,
    country VARCHAR(15) NOT NULL
);
```

```
-- Table 'evaluator'
CREATE TABLE evaluator (
    username VARCHAR(30),
    exp_years TINYINT(4) NOT NULL,
    firm CHAR(9),
    PRIMARY KEY (username),
    FOREIGN KEY (username) REFERENCES user(username),
    FOREIGN KEY (firm) REFERENCES etaireia(AFM)
);
```

```
-- Table 'employee'
CREATE TABLE employee (
    username VARCHAR(30) UNIQUE NOT NULL,
    bio TEXT,
    sistatikes VARCHAR(35),
    certificates VARCHAR(35),
```

```

    PRIMARY KEY (username),
    FOREIGN KEY (username) REFERENCES user(username)
);

CREATE TABLE languages (
    candid VARCHAR(30),
    lang SET('EN','FR','SP','GE','CH','GR'),
    PRIMARY KEY (candid, lang),
    FOREIGN KEY (candid) REFERENCES employee(username)
);

CREATE TABLE project (
    candid VARCHAR(30),
    num TINYINT(4) NOT NULL,
    descr TEXT NOT NULL,
    url VARCHAR(60) NOT NULL,
    PRIMARY KEY (candid, num),
    FOREIGN KEY (candid) REFERENCES employee(username)
);

-- Table 'job'
CREATE TABLE job (
    id INT(11) AUTO_INCREMENT PRIMARY KEY,
    start_date DATE NOT NULL,
    salary FLOAT,
    position VARCHAR(60) NOT NULL,
    edra VARCHAR(60) NOT NULL,
    evaluator VARCHAR(30),
    announce_date DATETIME NOT NULL,
    submission_date DATE NOT NULL,
    FOREIGN KEY (evaluator) REFERENCES evaluator(username)
);

-- Table 'applies'
CREATE TABLE applies (
    cand_username VARCHAR(30),
    job_id INT(11),
    PRIMARY KEY (cand_username, job_id),
    FOREIGN KEY (cand_username) REFERENCES employee(username),
    FOREIGN KEY (job_id) REFERENCES job(id)
);

```

```

-- Table 'subject'
CREATE TABLE subject (
    title VARCHAR(36) PRIMARY KEY,
    descr TEXT,
    belongs_to VARCHAR(36),
    FOREIGN KEY (belongs_to) REFERENCES subject(title)
);

-- Table 'requires'
CREATE TABLE requires (
    job_id INT(11),
    subject_title VARCHAR(36),
    PRIMARY KEY (job_id, subject_title),
    FOREIGN KEY (job_id) REFERENCES job(id),
    FOREIGN KEY (subject_title) REFERENCES subject(title)
);

CREATE TABLE degree (
    titlos VARCHAR(150) NOT NULL,
    idryma VARCHAR(150) NOT NULL,
    bathmida ENUM('BSc', 'MSc', 'PhD') NOT NULL,
    PRIMARY KEY (titlos, idryma)
);

CREATE TABLE has_degree (
    degr_title VARCHAR(150) NOT NULL,
    degr_idryma VARCHAR(150) NOT NULL,
    cand_username VARCHAR(30) NOT NULL,
    etos YEAR(4) NOT NULL,
    grade FLOAT NOT NULL,
    PRIMARY KEY (degr_title, degr_idryma, cand_username),
    FOREIGN KEY (degr_title, degr_idryma) REFERENCES degree(titlos, idryma),
    FOREIGN KEY (cand_username) REFERENCES employee(username)
);

CREATE TABLE evaluation (
    cand_username VARCHAR(30),
    job_id INT(11),
    evaluator1 VARCHAR(30),
    evaluator2 VARCHAR(30),
    score1 TINYINT(4),
    score2 TINYINT(4),
    PRIMARY KEY (cand_username, job_id),

```

```

    FOREIGN KEY (cand_username) REFERENCES employee(username),
    FOREIGN KEY (job_id) REFERENCES job(id)
);

CREATE TABLE application_history (
    cand_username VARCHAR(30),
    job_id INT(11),
    eval1 VARCHAR(30),
    eval2 VARCHAR(30),
    status ENUM('completed', 'cancelled') NOT NULL,
    grade TINYINT(4),
    PRIMARY KEY (cand_username, job_id)
);

CREATE TABLE `application_history2` (
    `cand_username` varchar(30) NOT NULL,
    `job_id` int(11) NOT NULL,
    `status` enum('active','completed','cancelled') NOT NULL DEFAULT 'active',
    `application_date` date NOT NULL DEFAULT curdate()
);

CREATE TABLE dba (
    dba_username VARCHAR(30) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    FOREIGN KEY (dba_username) REFERENCES user(username)
);

CREATE TABLE dba_log (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    dba_username VARCHAR(30) NOT NULL,
    action VARCHAR(255) NOT NULL,
    log_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dba_username) REFERENCES dba(dba_username)
);

ALTER TABLE evaluation
ADD CONSTRAINT chk_score1_range CHECK (score1 BETWEEN 1 AND 20),
ADD CONSTRAINT chk_score2_range CHECK (score2 BETWEEN 1 AND 20);

ALTER TABLE applies
ADD COLUMN status ENUM('active', 'completed', 'cancelled') NOT NULL DEFAULT 'active';

```

```
ALTER TABLE applies
ADD COLUMN application_date DATE NOT NULL DEFAULT CURRENT_DATE;
```

2. Νέες απαιτήσεις

Για την επέκταση των νέων μηχανισμών, κατασκευάστηκαν διάφορες συναρτήσεις, procedures, triggers αλλά και επεκτάθηκαν οι ήδη υπάρχοντες πίνακες. Ακολουθούν παραδείγματα για το κάθε ερώτημα με λεπτομερή περιγραφή.

3.1.2.1

Για την εισαγωγή και διαχείριση των αιτήσεων προστέθηκαν δύο νέα πεδία στον πίνακα applies. Συγκεκριμένα προστέθηκε η στήλη status στην οποία θα καταγράφεται η κατάσταση κάθε αίτησης (ενεργή, ολοκληρωθείσα, ακυρωμένη) καθώς και μία νέα στήλη (application_date) στην οποία θα σημειώνεται η τρέχουσα ημερομηνία κατά την οποία δημιουργείται η κάθε νέα αίτηση.

```
ALTER TABLE applies
ADD COLUMN status ENUM('active', 'completed', 'cancelled') NOT NULL DEFAULT
'active';

ALTER TABLE applies
ADD COLUMN application_date DATE NOT NULL DEFAULT CURRENT_DATE;
```

Κατά την εγγραφή μιας νέας αίτησης, πρέπει να ελεγχθούν δύο προϋποθέσεις:

(α) Η αίτηση υποβάλλεται σε χρονικό διάστημα το οποίο δεν υπερκαλύπτει το διάστημα 15 ημερών πριν από την ημερομηνία έναρξης της θέσης. Με άλλα λόγια εάν μία αίτηση ξεκινάει στις 16 του τρέχοντος μήνα ο υποψήφιος μπορεί να υποβάλει αίτηση μέχρι και την πρώτη ημέρα. (β) Ο εργαζόμενος δεν πρέπει να έχει ήδη τρεις ενεργές αιτήσεις και (γ) δεν θα είναι δυνατό να ακυρωθεί μία αίτηση 10 μέρες πριν την έναρξη της θέσης. Το ερώτημα διαχωρίστηκε σε δύο procedures με σκοπό την εύρωστη λειτουργία.

Συνεπώς για την εισαγωγή νέας αίτησης κατασκευάστηκε η procedure [register_application](#) η οποία εισάγει μία νέα αίτηση εάν πληρούνται οι παρακάτω προϋποθέσεις

- Η αίτηση πρέπει να υποβληθεί έως και 15 ημέρες πριν από την ημερομηνία έναρξης της εργασίας.
- Ο υποψήφιος πρέπει να έχει λιγότερες από τρεις ενεργές αιτήσεις.

Συγκεκριμένα οι ανωτέρω προϋποθέσεις ελέγχονται στην παρακάτω συνθήκη

```
IF (active_applications >= 3) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot register application: Candidate has 3 active
applications.';
    -- Check if the current date is not within the allowed time frame to submit
the application
ELSEIF (CURDATE() > threshold_date) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot register application: Application period
expired!';
ELSE
```

Τότε εάν πληρούνται και τα δύο κριτήρια εισάγονται στον πίνακα applies νέα πεδία για κάθε αίτηση.

Για την ακύρωση αιτήσεων δημιουργήθηκε η procedure [cancel_application](#) η οποία έχει σχεδιαστεί για να ακυρώσει μια αίτηση εργασίας υπό την προϋπόθεση ότι θα ακυρωθεί έως και 10 ημέρες πριν από την ημερομηνία έναρξης της εργασίας. Εάν η συνθήκη δεν πληρούται, εμφανίζεται μήνυμα σφάλματος.

Ακολουθούν περιπτώσεις κλήσης των δύο procedures

```
MariaDB [erec]> call register_application('mariak',1);
ERROR 1644 (45000): Cannot register application: Application period expired!
MariaDB [erec]> select * from job where id=1;
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | start_date | salary | position | edra | evaluator | announce_date | submission_date |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 2024-02-01 | 55000 | Software Developer | Athens | johnv | 2024-01-15 09:00:00 | 2024-03-01 |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [erec]> select curdate();
+-----+
| curdate() |
+-----+
| 2024-01-19 |
+-----+
1 row in set (0.000 sec)
```


Όπως φαίνεται από το παραπάνω στιγμιότυπο καλείται η αίτηση για την εργασία με id 1 και αφού η σημερινή ημερομηνία είναι λιγότερο από 15 μέρες για την αρχή της εργασίας απορρίπτεται.

Σε περίπτωση που η αίτηση είναι αρκετά μακριά (χρονικά) από την έναρξη τότε ολοκληρώνεται επιτυχώς.

```
MariaDB [erec]> call register_application('mariak',2);
Query OK, 3 rows affected (0.003 sec)

MariaDB [erec]> select * from job where id=2;
+----+-----+-----+-----+-----+-----+
| id | start_date | salary | position | edra |
+----+-----+-----+-----+-----+
| 2  | 2024-03-01 | 45000  | Marketing Specialist | Thessaloniki |
+----+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

Σε περίπτωση που ο υποψήφιος έχει 3 ενεργές αιτήσεις τότε το εμφανίζεται ανάλογο μήνυμα

```
MariaDB [erec]> call register_application('mariak',4);
ERROR 1644 (45000): Cannot register application: Candidate has 3 active applications.
MariaDB [erec]> select * from applies where cand_username='mariak';
+-----+-----+-----+-----+
| cand_username | job_id | status | application_date |
+-----+-----+-----+-----+
| mariak        | 1      | active | 2024-01-19       |
| mariak        | 2      | active | 2024-01-19       |
| mariak        | 6      | active | 2024-01-19       |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

Για την ακύρωση της αίτησης τότε εάν η εργασία ξεκινάει σε λιγότερο από 10 μέρες τότε εμφανίζεται μήνυμα λάθους. Ειδάλλως η αίτηση ακυρώνεται επιτυχώς

```

MariaDB [erec]> CALL cancel_application('mariak', 1);
ERROR 1644 (45000): Cannot cancel application: Deadline exceeded
MariaDB [erec]> select * from job where id=1;
+----+-----+-----+-----+-----+-----+
| id | start_date | salary | position          | edra   | evaluator |
+----+-----+-----+-----+-----+-----+
| 1  | 2024-01-24 | 55000 | Software Developer | Athens | johnv     |
+----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [erec]> CALL cancel_application('mariak', 2);
Query OK, 2 rows affected (0.015 sec)

MariaDB [erec]> select * from applies where job_id=2;
+-----+-----+-----+-----+
| cand_username | job_id | status   | application_date |
+-----+-----+-----+-----+
| mariak        | 2      | cancelled | 2024-01-19       |
| nickp         | 2      | active   | 2024-01-19       |
+-----+-----+-----+-----+
2 rows in set (0.000 sec)

```

3.1.2.2

Για το συγκεκριμένο ερώτημα δημιουργήθηκε ο πίνακας `evaluation` που περιέχει στοιχεία για τις αξιολογήσεις μαζί με ονόματα αξιολογητών αλλά και βαθμολογίες. Επίσης με την εντολή

```

ALTER TABLE evaluation
ADD CONSTRAINT chk_score1_range CHECK (score1 BETWEEN 1 AND 20),
ADD CONSTRAINT chk_score2_range CHECK (score2 BETWEEN 1 AND 20);

```

Εξασφαλίζεται ότι οι βαθμολογίες των αξιολογητών θα είναι στο διάστημα 1-20.

```
MariaDB [erec]> describe evaluation;
```

Field	Type	Null	Key	Default	Extra
cand_username	varchar(30)	NO	PRI	NULL	
job_id	int(11)	NO	PRI	NULL	
evaluator1	varchar(30)	YES		NULL	
evaluator2	varchar(30)	YES		NULL	
score1	tinyint(4)	YES		NULL	
score2	tinyint(4)	YES		NULL	

```
6 rows in set (0.012 sec)
```

Επίσης δημιουργήθηκε ο πίνακας application_history2 όπου εισάγονται οι επιτυχημένες αιτήσεις.

```
MariaDB [erec]> describe application_history2;
```

Field	Type	Null	Key	Default	Extra
cand_username	varchar(30)	NO		NULL	
job_id	int(11)	NO		NULL	
status	enum('active', 'completed', 'cancelled')	NO		active	
application_date	date	NO		curdate()	

```
4 rows in set (0.011 sec)
```

Τέλος στο σώμα της procedure process_singe_application όπως αυτή περιγράφεται στο [ερώτημα3_1_3_3](#) δημιουργείται προσωρινός πίνακας temp_results στον οποίον κρατούνται οι βαθμολογίες των υποψηφίων για την θέση που δίνεται στην procedure process_single_application. Όλη η απαίτηση υλοποιείται στο πλαίσιο υλοποίησης της procedure που περιγράφεται στο [ερώτημα3_1_3_3](#)

3.1.2.3.

Ο πίνακας για το ιστορικό αιτήσεων είναι:

```
CREATE TABLE `application_history` (
  `cand_username` varchar(30) NOT NULL,
  `job_id` int(11) NOT NULL,
```

```

`eval1` varchar(30) DEFAULT NULL,
`eval2` varchar(30) DEFAULT NULL,
`status` enum('completed','cancelled') NOT NULL,
`grade` tinyint(4) DEFAULT NULL,
PRIMARY KEY (`cand_username`,`job_id`),
KEY `idx_grade` (`grade`),
KEY `idx_eval1` (`eval1`),
KEY `idx_eval2` (`eval2`)
);

```

Στη συνέχεια δημιουργείται procedure `populate_application_history()`, η οποία για πλήθος εγγραφών ~60000 εισάγει στον ανωτέρω πίνακα εγγραφές με τα στοιχεία που ζητήθηκαν. Ακολουθεί screenshot απο το User Interface που δημιουργήθηκε που δείχνει στιγμιότυπο από τις εγγραφές του συγκεκριμένου πίνακα.

Database Management App

application history						
	cand_username	job_id	eval1	eval2	status	grade
	employee70	987	evaluator8	evaluator2	completed	17
	employee70	988	evaluator9	evaluator9	completed	18
	employee70	989	evaluator10	evaluator5	completed	18
	employee70	992	evaluator3	evaluator6	completed	18
	employee70	995	evaluator4	evaluator2	completed	18
	employee70	999	evaluator3	evaluator5	completed	19
	employee71	3	evaluator7	evaluator6	completed	18
	employee71	4	evaluator4	evaluator2	completed	19
	employee71	6	evaluator3	evaluator6	completed	19
	employee71	9	evaluator1	evaluator2	completed	19
	employee71	12	evaluator4	evaluator2	completed	19

Ο κώδικας της procedure

```
DELIMITER //
CREATE PROCEDURE populate_application_history()
BEGIN
    DECLARE i INT DEFAULT 0;
    WHILE i < 90000 DO
        INSERT INTO application_history(eval1, eval2, cand_username, job_id,
status, grade)
        VALUES (
            CONCAT('evaluator', FLOOR(RAND() * 10)),
            CONCAT('evaluator', FLOOR(RAND() * 10)),
            CONCAT('employee', FLOOR(RAND() * 100)),
            FLOOR(RAND() * 1000) + 1,
            'completed',
            FLOOR(RAND() * 20) + 1
        )
        ON DUPLICATE KEY UPDATE
            grade = VALUES(grade);
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;
```

Όπως φαίνεται από το παρακάτω στιγμιότυπο οι εγγραφές στον πίνακα είναι ~60000

```
+-----+
| Eggrafes |
+-----+
|      59324 |
+-----+
1 row in set (0.032 sec)
```

3.1.2.4.

Για το συγκεκριμένο ερώτημα αρκεί να δημιουργηθούν δύο πίνακες ο dba και ο dba_log

```

CREATE TABLE dba (
    dba_username VARCHAR(30) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    FOREIGN KEY (dba_username) REFERENCES user(username)
);

CREATE TABLE dba_log (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    dba_username VARCHAR(30) NOT NULL,
    action VARCHAR(255) NOT NULL,
    log_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dba_username) REFERENCES dba(dba_username)
);

```

Στη συνέχεια κατασκευάζονται οι triggers που αντιστοιχούν στο ερώτημα 3.1.4.1

```

DELIMITER //

CREATE TRIGGER after_job_insert
AFTER INSERT ON job
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'root',
        CONCAT('Inserted new job with ID ', NEW.id),
        NOW()
    );
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_job_update
AFTER UPDATE ON job
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',    -- Extracts the username part
        CONCAT('Updated job with ID ', OLD.id),

```

```

        NOW()
    );
END //

DELIMITER ;
DELIMITER //

CREATE TRIGGER after_job_delete
AFTER DELETE ON job
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Deleted job with ID ', OLD.id),
        NOW()
    );
END //

DELIMITER ;

```

Με την ίδια λογική έχουν κατασκευαστεί και οι triggers για τους δύο ακόμα πίνακες που παρατίθενται στο παράρτημα. Ακολουθεί παράδειγμα λειτουργίας μετά από update στον πίνακα job.

```

MariaDB [erec]> update job set salary=55000 where edra='Athens';
Query OK, 0 rows affected (0.004 sec)
Rows matched: 3  Changed: 0  Warnings: 0

MariaDB [erec]> select *from dba_log;
+-----+-----+-----+-----+
| log_id | dba_username | action                                | log_timestamp          |
+-----+-----+-----+-----+
|      4 | alexz        | Updated job with ID 1 | 2024-01-19 01:51:28 |
|      5 | alexz        | Updated job with ID 4 | 2024-01-19 01:51:28 |
|      6 | alexz        | Updated job with ID 7 | 2024-01-19 01:51:28 |
+-----+-----+-----+-----+
3 rows in set (0.000 sec)

```

Όπως φαίνεται ο συγκεκριμένος πίνακας έχει 3 εγγραφές καθώς ενημερώθηκε το πεδίο salary του πίνακα job που αντιστοιχεί στην έδρα Αθήνα, με αποτέλεσμα 3 εγγραφές του να πάρουν διαφορετική τιμή και να καταγραφούν οι συγκεκριμένες ενέργειες στο πίνακα log.

3.1.3.1

Για τη συγκεκριμένη procedure πάρθηκε η εξής παραδοχή: Θεωρήθηκε ο πίνακας evaluation που έχει τα παρακάτω πεδία:

- Username υποψηφίου
- job id
- evaluator 1
- evaluator 2
- score 1
- score 2

Δηλαδή θα κρατούνται στον πίνακα evaluation τα ανωτέρω πεδία ώστε για κάθε υποψήφιο που κάνει αίτηση για συγκεκριμένη εργασία να υπάρχει το όνομα και ο βαθμός δύο αξιολογητών. Στη συνέχεια η procedure θα ανατρέχει στον πίνακα και σε περίπτωση που υπάρχει ο βαθμός από το συγκεκριμένο αξιολογητή θα τον εμφανίζει. Εάν δεν υπάρχει θα υπολογίζεται βάσει του δοθέντος πίνακα. Ακολουθεί στιγμιότυπο με τις περιπτώσεις χρήσης

```
MariaDB [erec]> CALL check_and_calculate_evaluation('dimg', 'nickp', 1, @gr);
Query OK, 1 row affected (0.001 sec)

MariaDB [erec]> SELECT @gr;
+-----+
| @gr |
+-----+
|  15 |
+-----+
1 row in set (0.000 sec)

MariaDB [erec]> select * from evaluation where cand_username='nickp';
+-----+-----+-----+-----+-----+-----+
| cand_username | job_id | evaluator1 | evaluator2 | score1 | score2 |
+-----+-----+-----+-----+-----+-----+
| nickp        |      1 | dimg      | sophiaM    |      15 |      16 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)
```



```

MariaDB [erec]> select * from evaluation where cand_username='nickp';
+-----+-----+-----+-----+-----+-----+
| cand_username | job_id | evaluator1 | evaluator2 | score1 | score2 |
+-----+-----+-----+-----+-----+-----+
| nickp        |      1 | dimg       | sophiaM    |      15 |      16 |
| nickp        |      2 | johnv      | sophiaM    |     NULL |      18 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)

MariaDB [erec]> CALL check_and_calculate_evaluation('johnv', 'nickp', 2, @gr);
Query OK, 4 rows affected (0.001 sec)

MariaDB [erec]> select @gr;
+-----+
| @gr |
+-----+
|    7 |
+-----+

MariaDB [erec]> CALL check_and_calculate_evaluation('sophiaM', 'nickp', 2, @gr);
Query OK, 1 row affected (0.000 sec)

MariaDB [erec]> select @gr;
+-----+
| @gr |
+-----+
|   18 |
+-----+
1 row in set (0.000 sec)

```

Όπως φαίνεται από τα παραπάνω στιγμιότυπα για τον υποψήφιο που λείπει ένας βαθμός υπολογίζεται βάσει των προσόντων του ενώ στην περίπτωση που ο βαθμός δεν είναι κενός (score2) τότε εμφανίζεται.

3.1.3.2

Δημιουργήθηκε procedure `manage_job_application` που δοθέντος ενός εργαζομένου, μιας θέσης εργασίας και του χαρακτήρα δημιουργεί μία αίτηση, ακυρώνει αίτηση ή ενεργοποιεί ακυρωμένη αίτηση. Ακολουθούν παραδείγματα εκτέλεσης.

```

MariaDB [erec]> select * from evaluation;
+-----+-----+-----+-----+-----+-----+
| cand_username | job_id | evaluator1 | evaluator2 | score1 | score2 |
+-----+-----+-----+-----+-----+-----+
| mariak       | 6     | dimg      | johnv      | 12     | 14     |
| nickp        | 1     | dimg      | sophiaM    | 15     | 16     |
| nickp        | 2     | johnv     | sophiaM    | 15     | 18     |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

MariaDB [erec]> select * from applies;
+-----+-----+-----+-----+
| cand_username | job_id | status      | application_date |
+-----+-----+-----+-----+
| alexz         | 4     | active      | 2024-01-19       |
| chrisv        | 5     | active      | 2024-01-19       |
| elenid        | 1     | active      | 2024-01-19       |
| georgek       | 3     | active      | 2024-01-19       |
| mariak        | 3     | cancelled   | 2024-01-19       |
| nickp         | 2     | completed   | 2024-01-19       |
| nickp         | 7     | active      | 2024-01-19       |
+-----+-----+-----+-----+
7 rows in set (0.000 sec)

```

Όπως φαίνεται από το παραπάνω στιγμιότυπο υπάρχει evaluation για την αίτηση 6 της mariak αλλά δεν υπάρχει στον πίνακα applies. Καλώντας την procedure δημιουργείται η εγγραφή

```

MariaDB [erec]> call manage_job_application('mariak',6,'i');
+-----+
| Application created successfully |
+-----+
| Application created successfully |
+-----+
1 row in set (0.015 sec)

Query OK, 5 rows affected (0.018 sec)

MariaDB [erec]> select * from applies;
+-----+-----+-----+-----+
| cand_username | job_id | status      | application_date |
+-----+-----+-----+-----+
| alexz         | 4     | active      | 2024-01-19       |
| chrisv        | 5     | active      | 2024-01-19       |
| elenid        | 1     | active      | 2024-01-19       |
| georgek       | 3     | active      | 2024-01-19       |
| mariak        | 3     | cancelled   | 2024-01-19       |
| mariak        | 6     | active      | 2024-01-21       |
| nickp         | 2     | completed   | 2024-01-19       |
| nickp         | 7     | active      | 2024-01-19       |
+-----+-----+-----+-----+
8 rows in set (0.000 sec)

```

Σε περίπτωση που δεν υπάρχει η αξιολόγηση για τη θέση που πάμε να δημιουργήσουμε τότε η procedure βγάζει ανάλογο μήνυμα, ενημερώνει την αξιολόγηση προσθέτοντας εγγραφή για την αξιολόγηση της θέσης που εισάγουμε και έπειτα προτρέπει να επανεκτελεστεί η procedure για την εισαγωγή της αίτησης. Ακολουθούν λεπτομερή στιγμιότυπα της συγκεκριμένης περίπτωσης.

```
MariaDB [erec]> select * from evaluation;
+-----+-----+-----+-----+-----+-----+
| cand_username | job_id | evaluator1 | evaluator2 | score1 | score2 |
+-----+-----+-----+-----+-----+-----+
| nickp        | 1     | dimg       | sophiaM    | 15     | 16     |
| nickp        | 2     | johnv      | sophiaM    | 15     | 18     |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.000 sec)
```

```
MariaDB [erec]> select * from applies;
+-----+-----+-----+-----+
| cand_username | job_id | status      | application_date |
+-----+-----+-----+-----+
| alexz         | 4     | active      | 2024-01-19       |
| chrisv        | 5     | active      | 2024-01-19       |
| elenid        | 1     | active      | 2024-01-19       |
| georgek       | 3     | active      | 2024-01-19       |
| mariak        | 3     | cancelled   | 2024-01-19       |
| nickp         | 2     | completed   | 2024-01-19       |
| nickp         | 7     | active      | 2024-01-19       |
+-----+-----+-----+-----+
7 rows in set (0.000 sec)
```

```
MariaDB [erec]> call manage_job_application('mariak',6,'i');
+-----+
| Evaluators updated, please reapply |
+-----+
| Evaluators updated, please reapply |
+-----+
1 row in set (0.003 sec)
```

Query OK, 2 rows affected (0.006 sec)

```
MariaDB [erec]> select * from evaluation;
+-----+-----+-----+-----+-----+-----+
| cand_username | job_id | evaluator1 | evaluator2 | score1 | score2 |
+-----+-----+-----+-----+-----+-----+
| mariak        | 6     | dimg       | sophiaM    | NULL    | NULL    |
| nickp         | 1     | dimg       | sophiaM    | 15     | 16     |
| nickp         | 2     | johnv      | sophiaM    | 15     | 18     |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)
```

Όπως παρατηρείται πάμε να εισάγουμε αίτηση για μη αξιολογημένη θέση και συνεπώς εισάγεται η εγγραφή στην αξιολόγηση

Στη συνέχεια ξανακαλείται η procedure και η αίτηση εισάγεται επιτυχώς στο πίνακα applies.

```
MariaDB [erec]> call manage_job_application('mariak',6,'i');
+-----+
| Application created successfully |
+-----+
| Application created successfully |
+-----+
1 row in set (0.002 sec)

Query OK, 5 rows affected (0.006 sec)

MariaDB [erec]> select * from applies;
+-----+-----+-----+-----+
| cand_username | job_id | status   | application_date |
+-----+-----+-----+-----+
| alexz        | 4      | active   | 2024-01-19       |
| chrisv       | 5      | active   | 2024-01-19       |
| elenid       | 1      | active   | 2024-01-19       |
| georgek      | 3      | active   | 2024-01-19       |
| mariak       | 3      | cancelled| 2024-01-19       |
| mariak       | 6      | active   | 2024-01-21       |
| nickp       | 2      | completed| 2024-01-19       |
| nickp       | 7      | active   | 2024-01-19       |
+-----+-----+-----+-----+
8 rows in set (0.000 sec)
```

Σε περίπτωση που η αίτηση υπάρχει εμφανίζεται ανάλογο μήνυμα ενώ σε περίπτωση που η procedure κληθεί για να ακυρώσει αίτηση τότε εκτελείται και τυπώνεται κατάλληλο μήνυμα

```
MariaDB [erec]> call manage_job_application('mariak',6,'i');
+-----+
| Application already exists |
+-----+
| Application already exists |
+-----+
1 row in set (0.001 sec)

Query OK, 2 rows affected (0.002 sec)

MariaDB [erec]> call manage_job_application('mariak',6,'c');
+-----+
| Application cancelled successfully |
+-----+
| Application cancelled successfully |
+-----+
1 row in set (0.004 sec)
```

Για την κάλυψη της τρίτης προϋπόθεσης τότε εάν κληθεί με όρισμα 'a' ακυρώνεται η αίτηση ενώ εάν δεν υπάρχει αίτηση τότε εμφανίζεται κατάλληλο μήνυμα.

```
MariaDB [erec]> call manage_job_application('mariak',6,'a');
+-----+
| Application activated successfully |
+-----+
| Application activated successfully |
+-----+
1 row in set (0.003 sec)

Query OK, 3 rows affected (0.005 sec)

MariaDB [erec]> select * from applies;
+-----+-----+-----+-----+
| cand_username | job_id | status   | application_date |
+-----+-----+-----+-----+
| alexz        | 4      | active   | 2024-01-19       |
| chrisv       | 5      | active   | 2024-01-19       |
| elenid       | 1      | active   | 2024-01-19       |
| georgek      | 3      | active   | 2024-01-19       |
| mariak       | 3      | active   | 2024-01-19       |
| mariak       | 6      | active   | 2024-01-21       |
| nickp        | 2      | completed| 2024-01-19       |
| nickp        | 7      | active   | 2024-01-19       |
+-----+-----+-----+-----+
8 rows in set (0.000 sec)

MariaDB [erec]> call manage_job_application('mariak',4,'a');
+-----+
| No cancelled application to activate |
+-----+
| No cancelled application to activate |
+-----+
1 row in set (0.000 sec)
```

3.1.3.3

Για το συγκεκριμένο ερώτημα πάρθηκε η εξής παραδοχή: θεωρείται ότι οι αξιολογητές περνάνε από μία διαδικασία αξιολόγησης τις αιτήσεις και στις οποίες χειροκίνητα εισάγουν βαθμό σε κάθε υποψήφιο. Αυτή η διαδικασία γίνεται για κάθε υποψήφιο και θεωρείται ότι

για κάθε υποψήφιο που έχει αξιολογηθεί, η αξιολόγηση του έχει διεκπαιρωθεί και από τους δύο αξιολογητές. Συνεπώς σε περίπτωση που κάποιος αξιολογητής δεν έχει καταχωρήσει βαθμό η διαδικασία συνεχίζει στον υπολογισμό βαθμού βάσει κριτηρίων. Δημιουργήθηκε procedure `process_single_application` η οποία για κάθε υποψήφιο βάσει του δοθέντος `id` εργασίας επιλέγει το τελικό υποψήφιο (`top applicant`) βάσει του Μ.Ο των βαθμών από τους αξιολογητές. Εάν λείπει ένας βαθμός τότε υπολογίζει το βαθμό του βάσει των προσόντων του. Αυτή η προσέγγιση διασφαλίζει ότι κάθε αίτηση αξιολογείται με βάση έναν συνδυασμό υποκειμενικών αξιολογήσεων (από ανθρώπινους αξιολογητές) και αντικειμενικών κριτηρίων. Επισημαίνεται πώς οι υποψήφιοι που έχουν αξιολογηθεί και από τους δύο αξιολογητές επικρατούν έναντι αυτών που δεν έχουν έστω και ένα βαθμό. Ακολουθούν στιγμιότυπα από την εκτέλεση της.

Στο πρώτο παράδειγμα καλείται η procedure για μία εργασία στην οποία έχουν υποβληθεί πολλαπλές αιτήσεις. Μέσα από την ανωτέρω διαδικασία που περιγράφηκε επιλέγεται ο υποψήφιος που έχει βαθμό και από τους δύο αξιολογητές με τελικό βαθμό τον Μ.Ο.

```
MariaDB [erec]> call process_single_application(2);
```

Top_applicant	for job id:	a_job_id	with grade:	total_score
nickp		2		16.50

Για την επιβεβαίωση του αποτελέσματος εμφανίζονται και τα πεδία των συσχετιζόμενων πινάκων.

```

MariaDB [erec]> select * from evaluation where job_id=2;
+-----+-----+-----+-----+-----+-----+
| cand_username | job_id | evaluator1 | evaluator2 | score1 | score2 |
+-----+-----+-----+-----+-----+-----+
| nickp        |      2 | johnv      | sophiaM    |     15 |     18 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [erec]> select * from applies where job_id=2;
+-----+-----+-----+-----+
| cand_username | job_id | status      | application_date |
+-----+-----+-----+-----+
| mariak        |      2 | cancelled   | 2024-01-19       |
| nickp         |      2 | completed   | 2024-01-19       |
+-----+-----+-----+-----+
2 rows in set (0.000 sec)

```

Τέλος παρατηρείται ότι η αίτηση του έχει εισαχθεί στο ιστορικό με status completed.

```

MariaDB [erec]> select * from application_history2;
+-----+-----+-----+-----+
| cand_username | job_id | status      | application_date |
+-----+-----+-----+-----+
| nickp        |      2 | completed   | 2024-01-20       |
+-----+-----+-----+-----+
1 row in set (0.000 sec)

```

Στη συνέχεια καλώντας την για μία θέση όπου οι υποψήφιοι δεν έχουν αξιολογηθεί τότε το αποτέλεσμα βγαίνει βάσει του πίνακα που δίνεται.

```

MariaDB [erec]> select * from applies where job_id=3;
+-----+-----+-----+-----+
| cand_username | job_id | status | application_date |
+-----+-----+-----+-----+
| georgek      |      3 | active | 2024-01-19      |
| mariak       |      3 | active | 2024-01-19      |
+-----+-----+-----+-----+
2 rows in set (0.000 sec)

MariaDB [erec]> select * from evaluation where job_id=3;
Empty set (0.000 sec)

MariaDB [erec]> call process_single_application(3);
+-----+-----+-----+-----+-----+
| Top_applicant | for job id: | a_job_id | with grade: | total_score |
+-----+-----+-----+-----+-----+
| georgek      | for job id: |      3  | with grade: |      5.00   |
+-----+-----+-----+-----+-----+
1 row in set (0.015 sec)

```

Από το παραπάνω στιγμιότυπο φαίνεται ότι υπάρχουν δύο αιτήσεις για την θέση με id 3 οι οποίες δεν έχουν αξιολογηθεί. Τότε η procedure επιλέγει τον πρώτο υποψήφιο ο οποίος πληροί περισσότερα κριτήρια.

3.1.3.4

Στη συνέχεια για τη δημιουργία indexing στον πίνακα application_history εκτελούνται οι εξής εντολές που δημιουργούν indexes στις στήλες grade και evaluators

```

--a) --
CREATE INDEX idx_grade ON application_history (grade);
--b) --
CREATE INDEX idx_eval1 ON application_history (eval1);
CREATE INDEX idx_eval2 ON application_history (eval2);

```

Όπως φαίνεται από το παρακάτω ερώτημα χρησιμοποιείται το index στη στήλη του βαθμού.


```

MariaDB [erec]> explain SELECT cand_username, job_id
-> FROM application_history
-> WHERE grade BETWEEN 7 AND 12
-> ;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | application_history | range | idx_grade | idx_grade | 2 | NULL | 25142 | Using where; Using index |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

```

Όσον αφορά για τη δεύτερη procedure όπως φαίνεται παρακάτω χρησιμοποιήθηκε index στις στήλες των evaluators.

```

MariaDB [erec]> EXPLAIN SELECT cand_username, job_id
-> FROM application_history
-> WHERE eval1 = 'evaluatorUsername' OR eval2 = 'evaluatorUsername';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | application_history | index_merge | idx_eval1,idx_eval2 | idx_eval1,idx_eval2 | 123,123 | NULL | 2 | Using union(idx_eval1,idx_eval2); Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

```

Παρόλα αυτά παρατηρήθηκε ότι με τη χρήση του index τα query times είχαν μία βελτίωση της τάξης του ~5%. Συνεπώς κρίθηκε ως βέλτιστη επιλογή η χρήσης Union στα queries των εμφανίσεων. Επομένως το μέρος των procedures που εμφανίζει τους evaluators διαμορφώθηκε ως:

```

BEGIN
    SELECT cand_username, job_id FROM application_history WHERE eval1 =
evaluator_username
    UNION
    SELECT cand_username, job_id FROM application_history WHERE eval2 =
evaluator_username;
END

```

Τα αποτελέσματα ανέβασαν τη βελτιστοποίηση στο ~10%.

```

| employee99 | 989 |
| employee99 | 990 |
| employee99 | 997 |
| employee99 | 1000 |
+-----+-----+
9562 rows in set (0.010 sec)

Query OK, 0 rows affected (6.190 sec)

```

Κώδικας υλοποίησης κεφαλαίου 2

```

DELIMITER //

CREATE PROCEDURE register_application(
    IN p_cand_username VARCHAR(30),
    IN p_job_id INT(11)
)
BEGIN
    DECLARE job_start_date DATE;
    DECLARE active_applications INT;
    DECLARE threshold_date DATE;

    -- Retrieve the start date of the job
    SELECT start_date INTO job_start_date
    FROM job
    WHERE id = p_job_id;

    -- Calculate the threshold date which is 15 days before the job's start date
    SET threshold_date = DATE_SUB(job_start_date, INTERVAL 15 DAY);

    -- Count the number of active applications for the candidate
    SELECT COUNT(*) INTO active_applications
    FROM applies
    WHERE cand_username = p_cand_username AND status = 'active';

    -- Check if the candidate already has 3 active applications
    IF (active_applications >= 3) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot register application: Candidate has 3 active
applications.';
    -- Check if the current date is not within the allowed time frame to submit
the application
    ELSEIF (CURDATE() > threshold_date) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot register application: Application period
expired!';
    ELSE
        -- Insert the new application since all conditions are met
        INSERT INTO applies (cand_username, job_id, status, application_date)
        VALUES (p_cand_username, p_job_id, 'active', CURDATE());
    END IF;
END //

DELIMITER ;

```

```

DELIMITER //
CREATE PROCEDURE cancel_application(
    IN p_cand_username VARCHAR(30),
    IN p_job_id INT(11)
)
BEGIN
    DECLARE job_start_date DATE;

    -- Check the start date of the job
    SELECT start_date INTO job_start_date
    FROM job
    WHERE id = p_job_id;

    -- Check if condition is met
    IF (CURDATE() <= DATE_SUB(job_start_date, INTERVAL 10 DAY)) THEN
        -- Update the application status to 'cancelled'
        UPDATE applies
        SET status = 'cancelled'
        WHERE cand_username = p_cand_username AND job_id = p_job_id;
    ELSE
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot cancel application: Deadline exceeded';
    END IF;
END //
DELIMITER ;

-- -----3122-----
-- -- AT 3133!

-- ----- 3131-----

DELIMITER //

CREATE PROCEDURE check_and_calculate_evaluation(
    IN evaluator VARCHAR(30),
    IN employee_username VARCHAR(30),
    IN job_id INT,
    OUT grade TINYINT

```

```

)
BEGIN
    DECLARE degree_points INT DEFAULT 0;
    DECLARE language_points INT DEFAULT 0;
    DECLARE project_points INT DEFAULT 0;
    DECLARE existing_score1 TINYINT;
    DECLARE existing_score2 TINYINT;
    DECLARE ev1 VARCHAR(30);
    DECLARE ev2 VARCHAR(30);

    -- Check if an evaluation score has been registered by the evaluator
    SELECT score1, score2, evaluator1, evaluator2 INTO existing_score1,
existing_score2, ev1, ev2
    FROM evaluation
    WHERE (ev1 = evaluator) OR (ev2 = evaluator)
    AND cand_username = employee_username
    AND job_id = job_id;

    IF (existing_score1 IS NOT NULL AND evaluator = ev1) THEN
        SET grade = existing_score1;
    ELSEIF (existing_score2 IS NOT NULL AND evaluator = ev2) THEN
        SET grade = existing_score2;
    END IF;

    IF (existing_score1 IS NULL AND evaluator = ev1) OR (existing_score2 IS NULL
AND evaluator = ev2) THEN
        SELECT SUM(
            CASE
                WHEN d.bathmida = 'BSc' THEN 1
                WHEN d.bathmida = 'MSc' THEN 2
                WHEN d.bathmida = 'PhD' THEN 3
            END
        ) INTO degree_points
    FROM has_degree h
    INNER JOIN degree d ON h.degr_title = d.titlos AND h.degr_idryma = d.idryma
    INNER JOIN employee e ON h.cand_username = e.username
    INNER JOIN applies a ON e.username = a.cand_username
    WHERE h.cand_username = employee_username AND a.status='active';

    -- Calculate language points
    SELECT (LENGTH(lang) - LENGTH(REPLACE(lang, ',', ''))) + 1) INTO
language_points
    FROM languages
    WHERE candid = employee_username;

```

```

        -- Calculate project points
        SELECT COUNT(*) INTO project_points
        FROM project
        WHERE candid = employee_username;

        -- Sum up points
        SET grade = degree_points + language_points + project_points;
    END IF;

END //

DELIMITER ;

-- ----- 3132 -----
DELIMITER //

CREATE PROCEDURE `manage_job_application`(IN `p_employee_username` VARCHAR(30), IN `p_job_id` INT, IN
`p_action_char` CHAR(1))
BEGIN
    DECLARE v_applicationExists BOOLEAN DEFAULT FALSE;
    DECLARE v_applicationStatus VARCHAR(10);
    DECLARE v_evaluator1 VARCHAR(30);
    DECLARE v_evaluator2 VARCHAR(30);

    SELECT COUNT(*), status INTO v_applicationExists, v_applicationStatus
    FROM applies
    WHERE cand_usrname = p_employee_username AND job_id = p_job_id;

    SET v_applicationExists = v_applicationExists > 0;

    IF p_action_char = 'I' THEN

        SELECT evaluator1, evaluator2 INTO v_evaluator1, v_evaluator2
        FROM evaluation
        WHERE job_id = p_job_id;

```

```

IF v_evaluator1 IS NOT NULL AND v_evaluator2 IS NOT NULL THEN
    IF NOT v_applicationExists THEN
        INSERT INTO applies (cand_username, job_id, status) VALUES (p_employee_username, p_job_id, 'active');
        SELECT 'Application created successfully';
    ELSE
        SELECT 'Application already exists';
    END IF;
ELSE

    SET v_evaluator1 = (SELECT username FROM evaluator ORDER BY RAND() LIMIT 1);
    SET v_evaluator2 = (SELECT username FROM evaluator WHERE username <> v_evaluator1 ORDER BY
RAND() LIMIT 1);

    INSERT INTO evaluation VALUES(p_employee_username,p_job_id,v_evaluator1, v_evaluator2,NULL,NULL);
    SELECT 'Evaluators updated, please reapply';
    END IF;
ELSEIF p_action_char = 'c' THEN

    IF v_applicationExists AND v_applicationStatus = 'active' THEN
        UPDATE applies SET status = 'cancelled' WHERE cand_username = p_employee_username AND job_id =
p_job_id;
        SELECT 'Application cancelled successfully';
    ELSE
        SELECT 'No active application to cancel';
    END IF;
ELSEIF p_action_char = 'a' THEN

    IF v_applicationExists AND v_applicationStatus = 'cancelled' THEN
        UPDATE applies SET status = 'active' WHERE cand_username = p_employee_username AND job_id =
p_job_id;
        SELECT 'Application activated successfully';
    ELSE
        SELECT 'No cancelled application to activate';
    END IF;
ELSE

```

```

        SELECT 'Invalid action character';
    END IF;

END //
DELIMITER ;

-- ----- 3133 -----
DELIMITER //
CREATE PROCEDURE process_single_application(IN a_job_id INT)
BEGIN
    DROP TEMPORARY TABLE IF EXISTS temp_results;
    CREATE TEMPORARY TABLE IF NOT EXISTS temp_results (
        cand_username VARCHAR(30),
        total_score DECIMAL(5,2),
        application_date DATE,
        PRIMARY KEY (cand_username)
    );

    -- Fetch scores from evaluations and qualifications
    INSERT INTO temp_results (cand_username, total_score, application_date)
    SELECT
        a.cand_username,
        CASE
            WHEN ev.score1 IS NOT NULL AND ev.score2 IS NOT NULL THEN (ev.score1
+ ev.score2) / 2
            ELSE SUM(
                CASE
                    WHEN d.bathmida = 'PhD' THEN 3
                    WHEN d.bathmida = 'MSc' THEN 2
                    WHEN d.bathmida = 'BSc' THEN 1
                    ELSE 0
                END
            ) + (SELECT (LENGTH(lang) - LENGTH(REPLACE(lang, ',', '')) + 1)) +
COUNT(DISTINCT p.num)
        END AS total_score,
        a.application_date
    FROM applies a
    LEFT JOIN evaluation ev ON a.cand_username = ev.cand_username AND a.job_id =
ev.job_id
    LEFT JOIN has_degree hd ON a.cand_username = hd.cand_username
    LEFT JOIN degree d ON hd.degr_title = d.titlos AND hd.degr_idryma = d.idryma
    LEFT JOIN languages l ON a.cand_username = l.candid
    LEFT JOIN project p ON a.cand_username = p.candid

```

```

WHERE a.job_id = a_job_id AND a.status = 'active'
GROUP BY a.cand_username, a.application_date
ORDER BY total_score DESC, a.application_date ASC;

-- Debugging: Select all data from temp_results
SELECT cand_username as Top_applicant, 'for job id:', a_job_id , ' with
grade:', total_score
FROM temp_results;

-- Determine the candidate with the highest score
SET @winner := (SELECT cand_username FROM temp_results ORDER BY total_score
DESC, application_date ASC LIMIT 1);

-- Move the selected candidate's application to application_history2 and mark
as 'completed'
INSERT INTO application_history2 (top_applicant, job_id, status)
SELECT cand_username, job_id, 'completed'
FROM applies
WHERE job_id = a_job_id AND cand_username = @winner;

-- Update the status of the winning application to 'completed'
UPDATE applies
SET status = 'completed'
WHERE job_id = a_job_id AND cand_username = @winner;

-- Update other applications for the same job to 'cancelled'
UPDATE applies
SET status = 'cancelled'
WHERE job_id = a_job_id AND cand_username != @winner;

-- Cleanup: drop the temporary table
DROP TEMPORARY TABLE IF EXISTS temp_results;
END //

DELIMITER ;

-- -----3134-----
-- -----INDEXES-----
-- a) --
CREATE INDEX idx_grade ON application_history (grade);
-- b) --
CREATE INDEX idx_eval1 ON application_history (eval1);
CREATE INDEX idx_eval2 ON application_history (eval2);

```



```

-- a--

DELIMITER //
CREATE PROCEDURE get_applications_by_grade_range(
    IN min_grade TINYINT,
    IN max_grade TINYINT
)
BEGIN
    SELECT cand_username, job_id
    FROM application_history
    WHERE grade BETWEEN min_grade AND max_grade;
END //

DELIMITER ;

-- b--
-
DELIMITER //
CREATE PROCEDURE get_applications_by_evaluator(
    IN evaluator_username VARCHAR(30)
)
BEGIN
    SELECT cand_username, job_id FROM application_history WHERE eval1 =
evaluator_username
    UNION
    SELECT cand_username, job_id FROM application_history WHERE eval2 =
evaluator_username;
END //

DELIMITER ;

```

3. Κεφάλαιο 3 – Triggers

3.1.4.1

Το συγκεκριμένο ερώτημα υλοποιήθηκε στα πλαίσια της απαίτησης δημιουργίας log table όπως περιγράφεται στο [ερώτημα3_1_2_4](#)

Ο λεπτομερής κώδικας των 9 triggers βρίσκεται στο τέλος του κεφαλαίου.

3.1.4.2

Για τη διασφάλιση ότι δεν θα εισάγονται αιτήσεις λιγότερο από 15 μέρες πριν την έναρξη της θέσης αλλά και σε περίπτωση όπου ο εργαζόμενος δεν θα μπορεί να κάνει αίτηση όταν έχει ήδη τρεις ενεργές αιτήσεις δημιουργήθηκε trigger prevent_application_entry. Ακολουθούν δύο παραδείγματα εισαγωγής αιτήσεων.

```
MariaDB [erec]> select * from job;
```

id	start_date	salary	position	edra	evaluator	announce_date
1	2024-01-24	55000	Software Developer	Athens	johnv	2024-01-15 09:00:00
2	2024-03-01	45000	Marketing Specialist	Thessaloniki	dimg	2024-01-20 10:00:00
3	2024-04-01	65000	Project Manager	Patras	sophiaM	2024-01-25 11:00:00
4	2024-02-15	55000	Graphic Designer	Athens	johnv	2024-01-10 08:00:00
5	2024-03-20	52000	Data Analyst	Thessaloniki	dimg	2024-01-30 09:30:00
6	2024-05-01	70000	IT Manager	Patras	sophiaM	2024-02-15 10:00:00
7	2024-03-05	55000	HR Specialist	Athens	johnv	2024-02-01 08:45:00
8	2024-04-10	53000	Sales Executive	Thessaloniki	dimg	2024-02-20 11:00:00

```
8 rows in set (0.000 sec)
```

```
MariaDB [erec]> insert into applies values('nickp',4,'active','2024-02-10');  
ERROR 1644 (45000): Application cannot be submitted less than 15 days from the job start date
```

Όπως παρατηρείται στο ανωτέρω στιγμιότυπο η θέση με id 4 αρχίζει στις 15/02 οπότε κάθε αίτηση που γίνεται στο δεκαπενθήμερο πριν αρχίσει απορρίπτεται.

```

MariaDB [erec]> select * from applies;
+-----+-----+-----+-----+
| cand_username | job_id | status   | application_date |
+-----+-----+-----+-----+
| alexz        | 4      | active   | 2024-01-19       |
| chrisv       | 5      | active   | 2024-01-19       |
| elenid       | 1      | active   | 2024-01-19       |
| georgek      | 3      | active   | 2024-01-19       |
| mariak       | 3      | active   | 2024-01-19       |
| mariak       | 4      | active   | 2024-01-21       |
| mariak       | 6      | active   | 2024-01-21       |
| nickp       | 2      | completed | 2024-01-19       |
| nickp       | 7      | active   | 2024-01-19       |
+-----+-----+-----+-----+
9 rows in set (0.000 sec)

MariaDB [erec]> insert into applies values('mariak',7,'active',curdate());
ERROR 1644 (45000): Employee already has three active applications

```

Όπως παρατηρείται στο ανωτέρω στιγμιότυπο απορρίπτεται κάθε αίτηση της οποίας ο υποψήφιος έχει ήδη τρεις ενεργές ατήσεις.

3.1.4.3

Για την αποτροπή της ακύρωσης μίας αίτησης σε διάστημα λιγότερο των δέκα ημερών από την ημερομηνία έναρξης αλλά και για της ενεργοποίησης ακυρωθείσας αίτησης που ο υποψήφιος έχει ήδη τρεις ενεργές αιτήσεις δημιουργήθηκε trigger prevent_application_cancellation. Ακολουθεί παράδειγμα χρήσης του.

```

MariaDB [erec]> update applies set status='cancelled' where job_id=1;
ERROR 1644 (45000): Application cannot be cancelled less than 10 days from the job start date

```

Κώδικας υλοποίησης κεφαλαίου 3

```

----TRIGGERS----

DELIMITER //

CREATE TRIGGER after_job_insert
AFTER INSERT ON job
FOR EACH ROW

```

```
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Inserted new job with ID ', NEW.id),
        NOW()
    );
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_job_update
AFTER UPDATE ON job
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Updated job with ID ', OLD.id),
        NOW()
    );
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_job_delete
AFTER DELETE ON job
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Deleted job with ID ', OLD.id),
        NOW()
    );
END //

DELIMITER ;
```

```
DELIMITER //
```

```
CREATE TRIGGER after_user_insert
AFTER INSERT ON user
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Inserted new user with username ', NEW.username),
        NOW()
    );
END //
```

```
DELIMITER ;
```

```
DELIMITER //
```

```
CREATE TRIGGER after_user_update
AFTER UPDATE ON user
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Updated user with username ', OLD.username),
        NOW()
    );
END //
```

```
DELIMITER ;
```

```
DELIMITER //
```

```
CREATE TRIGGER after_user_delete
AFTER DELETE ON user
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Deleted user with username ', OLD.username),
        NOW()
    );
END //
```

```

END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_degree_insert
AFTER INSERT ON degree
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Inserted new degree with title ', NEW.titlos, ' at ',
NEW.idryma),
        NOW()
    );
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_degree_update
AFTER UPDATE ON degree
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)
    VALUES (
        'alexz',
        CONCAT('Updated degree with title ', OLD.titlos, ' at ', OLD.idryma),
        NOW()
    );
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_degree_delete
AFTER DELETE ON degree
FOR EACH ROW
BEGIN
    INSERT INTO dba_log (dba_username, action, log_timestamp)

```

```

VALUES (
    'alexz',
    CONCAT('Deleted degree with title ', OLD.titlos, ' from ', OLD.idryma),
    NOW()
);
END //

DELIMITER ;

-----3.1.4.2 -----
DELIMITER //

CREATE TRIGGER prevent_application_entry
BEFORE INSERT ON applies
FOR EACH ROW
BEGIN
    DECLARE application_count INT;

    -- Check if the submission date is less than 15 days from the start date
    SELECT start_date INTO @job_start_date FROM job WHERE id = NEW.job_id;
    IF NEW.application_date > DATE_SUB(@job_start_date, INTERVAL 15 DAY) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Application cannot be submitted less than 15 days from
the job start date';
    END IF;

    -- Check if the employee already has three active applications
    SELECT COUNT(*) INTO application_count
    FROM applies
    WHERE cand_username = NEW.cand_username AND status = 'active';

    IF application_count >= 3 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee already has three active applications';
    END IF;
END;
//

DELIMITER ;

```

```

-----3.1.4.3 -----
DELIMITER //

CREATE TRIGGER prevent_application_cancellation
BEFORE UPDATE ON applies
FOR EACH ROW
BEGIN
    DECLARE application_count INT;
    DECLARE job_start_date DATE;

    -- Check if the status is being updated to 'cancelled'
    IF NEW.status = 'cancelled' AND OLD.status <> 'cancelled' THEN
        -- Get the start date of the job associated with the application
        SELECT start_date INTO job_start_date FROM job WHERE id = NEW.job_id;

        -- Check if the date of cancellation is less than 10 days from the start date
        IF NEW.application_date >= DATE_SUB(job_start_date, INTERVAL 10 DAY) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Application cannot be cancelled less than 10 days from
the job start date';
        END IF;
    END IF;

    -- Check if the employee already has three active applications
    IF NEW.status = 'active' THEN
        SELECT COUNT(*) INTO application_count
        FROM applies
        WHERE cand_username = NEW.cand_username AND status = 'active';

        IF application_count >= 3 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Employee already has three active applications';
        END IF;
    END IF;
END;
//

DELIMITER ;

```


4. Graphical User Interface

Για την δημιουργία γραφικής διεπαφής για τη βάση, επιλέχθηκε το PyQt5 το οποίο πρόκειται για ένα ελαφρύ, λιτό και πλήρως λειτουργικό περιβάλλον σε Python που βασίζεται στο Qt παρέχοντας τη συνοχή και ευελιξία που προσφέρει το τελευταίο. Το PyQt δίνει τη δυνατότητα στους προγραμματιστές να δημιουργούν πλούσιες σε χαρακτηριστικά, εφαρμογές με ελκυστικά γραφικά και φιλικά προς τον χρήστη. Η ενσωμάτωσή του με την Python συνδυάζει τα ολοκληρωμένα χαρακτηριστικά του Qt με την απλότητα και την αναγνωσιμότητα της Python, καθιστώντας το μια δημοφιλή επιλογή για γρήγορη ανάπτυξη γραφικών διεπαφών.

Σε σύγκριση με άλλα GUI που προσφέρει η Java όπως το Swing, JavaFX, Eclipse το PyQt συχνά θεωρείται πιο φιλικό προς το χρήστη και ελαφρύ. Αυτό οφείλεται εν μέρει στο ότι η σύνταξη της Python είναι γενικά πιο συνοπτική και πιο ευανάγνωστη, και στο ότι το PyQt παρέχει μια διεπαφή που λειτουργεί απρόσκοπτα με υψηλή απόδοση.

Το πλεονέκτημα του PyQt είναι επίσης εμφανές στην υποστήριξη πληθώρας βάσεων δεδομένων. Είναι σημαντικό να αναφερθεί πώς η ανάπτυξη του κώδικα έγινε σε περιβάλλον macOS και mysql ενώ ενσωματώθηκε απροβλημάτιστα σε περιβάλλο windows με χρήση apache server και mariadb για την επέκτασή του. Αυτό διευκόλυνε ότι μπορέσαμε να επικεντρωθούμε περισσότερο στη λειτουργικότητα της εφαρμογής και λιγότερο στις περιπλοκές της επικοινωνίας της βάσης δεδομένων καθώς δουλεύαμε παράλληλα σε διαφορετικά λειτουργικά συστήματα.

Τελικά, το αποτέλεσμα της χρήσης του PyQt για την ανάπτυξη εφαρμογών είναι ένα φιλικό προς τον χρήστη, ελαφρύ και απλό περιβάλλον. Η απλότητα και η δύναμη της Python, σε συνδυασμό με τα ισχυρά χαρακτηριστικά του Qt, επιτρέπουν τη δημιουργία επαγγελματικών και οπτικά ελκυστικών εφαρμογών. Η ευκολία χρήσης, η συμβατότητα μεταξύ πλατφορμών και το ολοκληρωμένο σύνολο εργαλείων και δυνατοτήτων καθιστούν το PyQt εξαιρετική επιλογή.

Η Java, ενώ είναι ισχυρή και ευέλικτη, αντιμετωπίζει ορισμένες προκλήσεις στην ανάπτυξη GUI. Η περίπλοκη σύνταξη μπορεί να οδηγήσει σε μακροσκελείς και πολύπλοκες υλοποιήσεις κώδικα, καθιστώντας την ταχεία ανάπτυξη και συντήρηση πιο περίπλοκη. Τα Java GUI συχνά δυσκολεύονται να ταιριάξουν με την εγγενή εμφάνιση διαφορετικών λειτουργικών συστημάτων, με αποτέλεσμα ενδεχομένως λιγότερο ελκυστικές εφαρμογές. Επίσης ενδέχεται να προκύψουν προβλήματα απόδοσης.

Περιγραφή κώδικα και κλάσεων:

1. Imports and Initialization:

- Ο κώδικας ξεκινάει εισάγοντας τις απαραίτητες βιβλιοθήκες/ενότητες για την εκτέλεση της εφαρμογής.

2. CustomTableModel

- Δημιουργείται μία υποκλάση της κλάσης QAbstractTableModel, που παρέχει ένα αφηρημένο μοντέλο για τη δημιουργία μοντέλων πινάκων.
- Υλοποιούνται βασικές μέθοδοι όπως rowCount, columnCount, data για να καθοριστεί ο τρόπος αποθήκευσης των δεδομένων πίνακα, πόσες γραμμές/στήλες καθώς και η παρουσίαση των δεδομένων.
- Η headerData υλοποιείται για να παρέχει τίτλους για τις στήλες.

3. DynamicForm (QDialog):

- Αυτή η κλάση δημιουργεί μια δυναμική φόρμα που βασίζεται στη δομή του πίνακα της βάσης δεδομένων. Χρησιμοποιείται για την προσθήκη ή την επεξεργασία εγγραφών.
- Κληρονομείται από το QDialog, παρέχοντας ένα αναδυόμενο παράθυρο που μπορεί να δέχεται είσοδο.
- Η εμφάνιση της φόρμας προσαρμόζεται χρησιμοποιώντας το setStyleSheet με το οποίο καθορίζονται με CSS η εμφάνιση και η στοίχιση

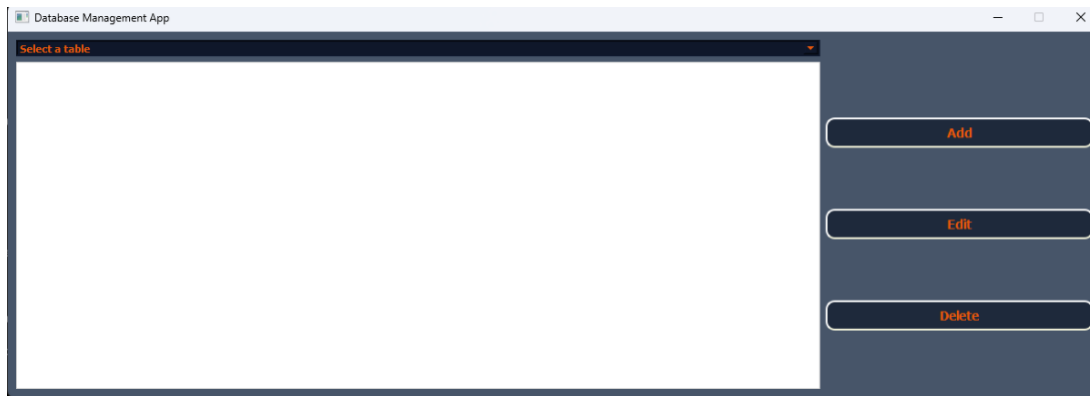
5. DatabaseApp (QMainWindow):

- Το κύριο παράθυρο της εφαρμογής, υποκλάση του QMainWindow.
- Αρχικοποιεί τη σύνδεση διεπαφής χρήστη και βάσης δεδομένων.
- Η μέθοδος initUI ρυθμίζει τη διάταξη, τα γραφικά στοιχεία (όπως κουμπιά, προβολή πινάκων κ.α) και την εμφάνιση τους. Οι λειτουργίες CRUD (Δημιουργία, Ανάγνωση, Ενημέρωση, Διαγραφή) υλοποιούνται σε αυτήν την κλάση (addRecord, editRecord, deleteRecord, refreshTable, κ.λπ.).
- Το getTableStructure ανακτά τη δομή (ονόματα στηλών και τύποι) ενός συγκεκριμένου πίνακα από τη βάση δεδομένων, ο οποίος χρησιμοποιείται για τη δυναμική δημιουργία φορμών.

5. Styling

- CSS εφαρμόζεται χρησιμοποιώντας τη μέθοδο setStyleSheet για την προσαρμογή της εμφάνισης διαφορετικών γραφικών στοιχείων (όπως QDialog, QLabel, QLineEdit, QPushButton).
- Τα QGridLayout και QVBoxLayout χρησιμοποιούνται για την τακτοποίηση γραφικών στοιχείων στο παράθυρο.
- Όταν εκτελείται ο κώδικας, δημιουργεί ένα στιγμιότυπο (instance) του QApplication, μετά το DatabaseApp και έτσι εκτελείται το GUI. Στα επόμενα στιγμιότυπα αποτυπώνονται οι λειτουργίες της γραφικής διεπαφής.

Αρχική εκτέλεση του παραθύρου:



Στην αρχή εκτελείται ο κώδικας και το παράθυρο που εμφανίζεται αποτελείται από ένα dropdown που προτρέπει το χρήστη να επιλέξει έναν πίνακα και δεξιά υπάρχουν 3 κουμπιά Add, Edit, Delete.

Επιλέγοντας έναν πίνακα εμφανίζονται οι εγγραφές του

Job							
id	start_date	salary	position	edra	evaluator	announce_date	submission_date
1	2024-01-24	55000	Software Developer	Athens	johnv	2024-01-15	2024-03-01
2	2024-03-01	45000	Marketing Specialist	Thessaloniki	dimg	2024-01-20	2024-04-01
3	2024-04-01	65000	Project Manager	Patras	sophiaM	2024-01-25	2024-05-01
4	2024-02-15	55000	Graphic Designer	Athens	johnv	2024-01-10	2024-03-15
5	2024-03-20	52000	Data Analyst	Thessaloniki	dimg	2024-01-30	2024-04-20
6	2024-05-01	70000	IT Manager	Patras	sophiaM	2024-02-15	2024-06-01
7	2024-03-05	55000	HR Specialist	Athens	johnv	2024-02-01	2024-04-05
8	2024-04-10	53000	Sales Executive	Thessaloniki	dimg	2024-02-20	2024-05-10

Στη συνέχεια μπορούμε να πατήσουμε το edit για να επεξεργαστούμε μία εγγραφή, όπου εμφανίζεται ένα νέο παράθυρο φόρμας για να επεξεργαστούμε τα στοιχεία της συγκεκριμένης εγγραφής

Αλλάζουμε το πεδίο και πατώντας ok αυτό αποτυπώνεται στον πίνακα.

id

2

start_date

2024-03-01

salary

50000.0

position

Marketing Specialist

edra

Thessaloniki

evaluator

ding

announce_date

2024-01-20 10:00:00

submission_date

2024-04-01

OK

Cancel

Database Management App

Job

id	start_date	salary	position	edra	evaluator	announce_date	submission_date
1	2024-01-24	55000	Software Developer	Athens	johnv	2024-01-15	2024-03-01
2	2024-03-01	50000	Marketing Specialist	Thessaloniki	ding	2024-01-20	2024-04-01
3	2024-04-01	65000	Project Manager	Patras	sophiaM	2024-01-25	2024-05-01
4	2024-02-15	55000	Graphic Designer	Athens	johnv	2024-01-10	2024-03-15
5	2024-03-20	52000	Data Analyst	Thessaloniki	ding	2024-01-30	2024-04-20
6	2024-05-01	70000	IT Manager	Patras	sophiaM	2024-02-15	2024-06-01
7	2024-03-05	55000	HR Specialist	Athens	johnv	2024-02-01	2024-04-05
8	2024-04-10	53000	Sales Executive	Thessaloniki	ding	2024-02-20	2024-05-10

Add

Edit

Delete

Πατώντας Add μπορούμε να προσθέσουμε μία εγγραφή

Database Management App

Evaluator

username	exp_years	firm
ding	10	234567891
johnv	5	123456789
sophiaM	8	345678912

Add

Edit

Delete

Erecruit Data

username

012

exp_years

4

firm

123456789

OK

Cancel

Database Management App

Evaluator

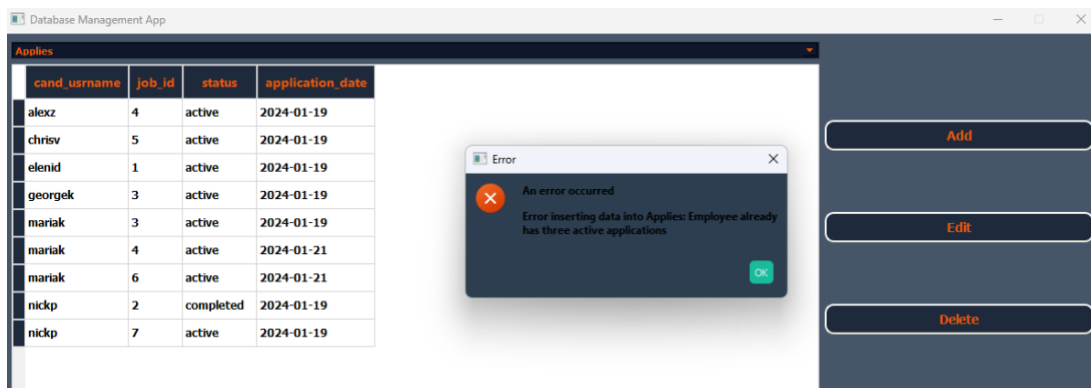
username	exp_years	firm
ding	10	234567891
johnv	5	123456789
012	4	123456789
sophiaM	8	345678912

Add

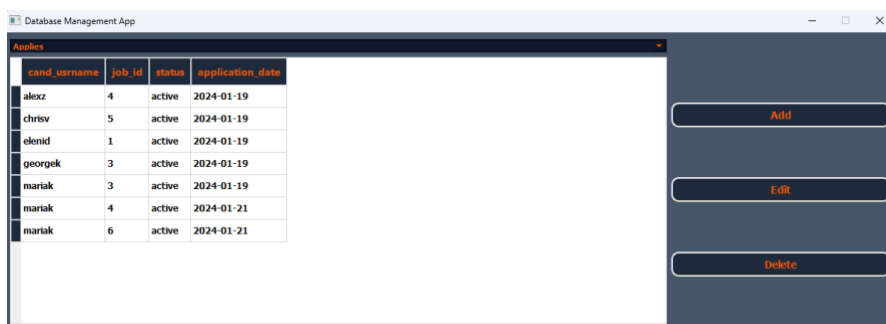
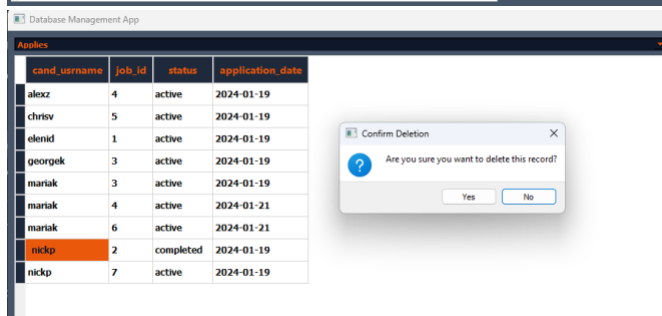
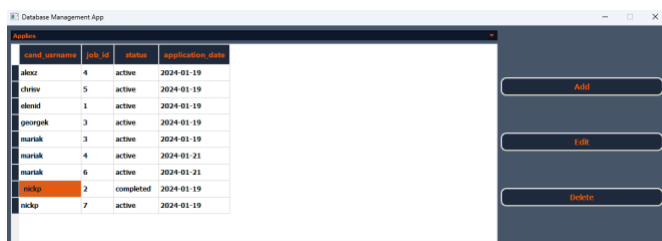
Edit

Delete

Για την προσπάθεια εισαγωγής στοιχείων που έχουν σύγκρουση με τις απαιτήσεις της βάσης και αποτρέπονται με κατάλληλους triggers εμφανίζεται μήνυμα προς το χρήστη



Τέλος επιλέγοντας μία εγγραφή και πατώντας delete τότε αυτή διαγράφεται επιτυχώς



Και ανανεώνεται αυτόματα το παράθυρο.

Κώδικας υλοποίησης κεφαλαίου 4

```
import sys
import mysql.connector
import datetime
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QComboBox, QTableView, QVBoxLayout, QWidget,
    QPushButton, QDialog, QLineEdit, QLabel, QDialogButtonBox, QMessageBox, QGridLayout
)
from PyQt5.QtCore import QAbstractTableModel, Qt

# Custom Table Model
class CustomTableModel(QAbstractTableModel):
    def __init__(self, data):
        super(CustomTableModel, self).__init__()
        self._data = data
        self.headers = []

    def rowCount(self, parent=None):
        return len(self._data)

    def columnCount(self, parent=None):
        return len(self.headers) if self._data else 0

    def data(self, index, role=Qt.DisplayRole):
        if role == Qt.DisplayRole:
            value = self._data[index.row()][index.column()]
            # Check if the value is a date instance and format it
            if isinstance(value, (datetime.date, datetime.datetime)):
                return value.strftime("%Y-%m-%d") # Format date as needed
            return value
        return None

    def headerData(self, section, orientation, role):
        if role == Qt.DisplayRole and orientation == Qt.Horizontal:
```

```
        if section < len(self.headers):
            return self.headers[section]
    return None
```

```
class DynamicForm(QDialog):
```

```
    def __init__(self, table_structure, parent=None):
        super(DynamicForm, self).__init__(parent)
        self.setWindowTitle('Erecruit Data')
```

```
    self.setStyleSheet("""
        QDialog {
            background-color: #1e293b;
        }
        QLabel {
            font-size: 14px;
            color: #cbd5e1;
            font-weight: bold;
        }
        QLineEdit {
            border: 1px solid #ccc;
            background-color: #17202e;
            padding: 5px;
            font-size: 14px;
            color: #ea580c;
            font-weight: bold;
        }
        QPushButton {
            background-color: #ea580c;
            color: #cbd5e1;
            font-weight: bold;
            padding: 6px;
            border-radius: 4px;
            font-size: 14px;
        }
        QPushButton:hover {
            background-color: #c44d27;
        }
    """)
```



```

        """

    layout = QVBoxLayout(self)

    self.inputs = {}
    for column, col_type, *_ in table_structure:
        self.inputs[column] = QLineEdit(self)
        layout.addWidget(QLabel(column))
        layout.addWidget(self.inputs[column])

    self.buttons = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel, self)
    self.buttons.accepted.connect(self.accept)
    self.buttons.rejected.connect(self.reject)
    layout.addWidget(self.buttons)

    def getInputs(self):
        return {column: self.inputs[column].text() for column in self.inputs}

class DatabaseApp(QMainWindow):
    def __init__(self):
        super().__init__()

        self.conn = self.createDatabaseConnection()
        self.initUI()

    def createDatabaseConnection(self):
        config = {
            "host": "localhost",
            "port": 3306,
            "user": "root",
            "password": "",
            "database": "PDB2024"
        }

        try:
            conn = mysql.connector.connect(**config)

```

```

        print("Successfully connected to the database!")
        return conn
    except mysql.connector.Error as e:
        self.showErrorDialog(f"Error connecting to MySQL: {e}")
        sys.exit(1)

def showErrorDialog(self, message):
    msgBox = QMessageBox()
    msgBox.setIcon(QMessageBox.Critical)
    msgBox.setText("An error occurred")
    msgBox.setInformativeText(message)
    msgBox.setWindowTitle("Error")
    # Custom styling
    msgBox.setStyleSheet("""
    QMessageBox {
        background-color: #2C3E50;
        color: orange;
        font-weight: bold;
    }
    QMessageBox QPushButton {
        background-color: #18BC9C;
        color: #EAECEE;
        border-radius: 5px;
        padding: 6px;
        margin: 4px;
    }
    QMessageBox QPushButton:hover {
        background-color: #1ABC9C;
    }
    QMessageBox QPushButton:pressed {
        background-color: #16A085;
    }
    """)

    msgBox.exec_()

```

```

def initUI(self):

    self.setWindowTitle('Database Management App')

    self.setGeometry(100, 100, 1000, 800)

    self.setFixedSize(1200, 400)

    self.setStyleSheet("QMainWindow { background-color: #475569; }")

    # Main layout
    gridLayout = QGridLayout()

    # Dropdown for table selection
    self.tableDropdown = QComboBox(self)

    self.tableDropdown.setStyleSheet("QComboBox { background-color: #0f172a; color:#ea580c; font-weight:bold;
})")

    self.tableDropdown.addItem("Select a table","User", "Etaireia", "Evaluator", "Employee", "Languages",
"Project", "Job", "Applies", "Subject", "Requires", "Degree", "Has_degree","application_history")

    self.tableDropdown.currentIndexChanged.connect(self.tableSelected)

    gridLayout.addWidget(self.tableDropdown, 0, 0)

    # Table view to display data
    self.tableView = QTableView(self)

    self.tableView.setStyleSheet("""
        QTableView {
            border: 1px solid #d4d4d4;
            selection-background-color: #a2d2ff;
            gridline-color: #d4d4d4;
            font-size: 12px;
            font-family: 'Helvetica';
            font-weight: bold;
        }
        QTableView::item {
            padding: 5px;
        }
        QHeaderView::section {
            background-color: #1e293b;
            padding: 4px;
            border: 1px solid #d4d4d4;
            font-size: 14px;

```

```

        font-weight: bold;
        color: #ea580c;
    }
    QTableView::item:selected {
        background-color: #ea580c;
        color: #0f172a;
    }
    """
)
gridLayout.addWidget(self.tableView, 1, 0,)

```

Button styles

```

button_style = """
QPushButton {
    background-color: #1e293b;
    color: #ea580c;
    border-style: outset;
    border-width: 2px;
    border-radius: 10px;
    border-color: beige;
    font: bold 14px;
    min-width: 10em;
    padding: 6px;
}
QPushButton:hover {
    background-color: #0f172a;
}
QPushButton:pressed {
    background-color: #1e293b;
    border-style: inset;
}
"""

```

```

buttonLayout = QVBoxLayout()

```

Create CRUD operation buttons

```

self.addButton = QPushButton('Add', self)
self.addButton.setStyleSheet(button_style)

```

```

self.addButton.clicked.connect(self.addRecord)
buttonLayout.addWidget(self.addButton)

self.editButton = QPushButton('Edit', self)
self.editButton.setStyleSheet(button_style)
self.editButton.clicked.connect(self.editRecord)
buttonLayout.addWidget(self.editButton)

self.deleteButton = QPushButton('Delete', self)
self.deleteButton.setStyleSheet(button_style)
self.deleteButton.clicked.connect(self.deleteRecord)
buttonLayout.addWidget(self.deleteButton)

# Button layout
buttonLayout = QVBoxLayout()
buttonLayout.addWidget(self.addButton)
buttonLayout.addWidget(self.editButton)
buttonLayout.addWidget(self.deleteButton)
gridLayout.addLayout(buttonLayout, 1, 1)

gridLayout.setColumnStretch(0, 3)
gridLayout.setColumnStretch(1, 1)

# Set the layout to the QWidget
centralWidget = QWidget()
centralWidget.setLayout(gridLayout)
self.setCentralWidget(centralWidget)

# Display the main window
self.show()

def getTableStructure(self, table_name):
    cursor = self.conn.cursor()
    cursor.execute(f"DESCRIBE {table_name}")
    return cursor.fetchall() # Returns a list of columns and their types

```

```

def getSelectedRowData(self):
    selected_index = self.tableView.currentIndex()
    if not selected_index.isValid():
        return None
    return selected_index.row()

def addRecord(self):
    selected_table = self.tableDropdown.currentText()
    table_structure = self.getTableStructure(selected_table)

    dialog = DynamicForm(table_structure, self)
    if dialog.exec_() == QDialog.Accepted:
        inputs = dialog.getInputs()
        # Insert data into the database
        self.insertData(selected_table, inputs)
        self.refreshTable()

def updateData(self, table_name, new_data, old_data):
    # Construct the UPDATE statement
    set_clause = ', '.join([f"{key} = %s" for key in new_data.keys()])
    primary_key_column = self.getTableStructure(table_name)[0][0]
    primary_key_value = old_data[0]

    query = f"UPDATE {table_name} SET {set_clause} WHERE {primary_key_column} = %s"
    values = list(new_data.values()) + [primary_key_value]

    try:
        cursor = self.conn.cursor()
        cursor.execute(query, values)
        self.conn.commit()
    except mysql.connector.Error as e:
        self.showErrorDialog(f"Error updating data in {table_name}: {e}")

def editRecord(self):
    row = self.getSelectedRowData()
    if row is None:

```

```

        return # No row selected

    selected_table = self.tableDropdown.currentText()
    table_structure = self.getTableStructure(selected_table)
    print("Table Structure:", table_structure)
    current_data = self.model._data[row]

    dialog = DynamicForm(table_structure, self)
    for i, column in enumerate(table_structure):
        dialog.inputs[column[0]].setText(str(current_data[i]))

    if dialog.exec_() == QDialog.Accepted:
        inputs = dialog.getInputs()
        self.updateData(selected_table, inputs, current_data)
        self.refreshTable()

def insertData(self, table_name, data):
    # Prepare and execute INSERT statement
    columns = ', '.join(data.keys())
    placeholders = ', '.join(['%s'] * len(data))
    values = tuple(data.values())

    query = f"INSERT INTO {table_name} ({columns}) VALUES ({placeholders})"
    try:
        cursor = self.conn.cursor()
        cursor.execute(query, values)
        self.conn.commit()
    except mysql.connector.Error as e:
        self.showErrorDialog(f"Error inserting data into {table_name}: {e}")

def deleteData(self, table_name, pk_column, pk_value):
    # Prepare the DELETE statement
    query = f"DELETE FROM {table_name} WHERE {pk_column} = %s"

    try:
        cursor = self.conn.cursor()
        cursor.execute(query, (pk_value,))

```

```

        self.conn.commit()

    except mysql.connector.Error as e:
        self.showErrorDialog(f"Error deleting data from {table_name}: {e}")

def deleteRecord(self):
    row = self.getSelectedRowData()
    if row is None:
        return # No row selected

    selected_table = self.tableDropdown.currentText()
    primary_key_column = self.getTableStructure(selected_table)[0][0]
    primary_key_value = self.model._data[row][0]

    reply = QMessageBox.question(self, 'Confirm Deletion',
                                "Are you sure you want to delete this record?",
                                QMessageBox.Yes | QMessageBox.No, QMessageBox.No)

    if reply == QMessageBox.Yes:
        self.deleteData(selected_table, primary_key_column, primary_key_value)
        self.refreshTable()

def refreshTable(self):
    selected_table = self.tableDropdown.currentText()
    data, headers = self.fetchDataFromDatabase(selected_table)
    self.model = CustomTableModel(data)
    self.model.headers = headers
    self.tableView.setModel(self.model)
    self.tableView.resizeColumnsToContents()

def tableSelected(self, index):
    table_name = self.tableDropdown.itemText(index)
    # Fetch data from the database for the selected table
    data, headers = self.fetchDataFromDatabase(table_name)
    # Set the headers and data for the CustomTableModel

```



```

self.model = CustomTableModel(data)
self.model.headers = headers
self.tableView.setModel(self.model)
# Update the table view to adjust columns to content
self.tableView.resizeColumnsToContents()

def fetchDataFromDatabase(self, table_name):
# Implement logic to fetch data from the database
    try:
        cursor = self.conn.cursor()
        cursor.execute(f"DESCRIBE {table_name}")
        # Fetch column headers
        headers = [column[0] for column in cursor.fetchall()]

        cursor.execute(f"SELECT * FROM {table_name}")
        # Fetch all results as a list of tuples
        rows = cursor.fetchall()
        return rows, headers # Return rows and headers
    except mysql.connector.Error as e:
        self.showErrorDialog(f"Error fetching data from {table_name}: {e}")
        return [], [] # Return empty lists on error

if __name__ == "__main__":
    app = QApplication(sys.argv)
    ex = DatabaseApp()
    font_family = "Helvetica"
    sys.exit(app.exec_())

```